

Your proposal describes a system that is **not just another chatbot or RAG application**. It is an **AI-powered Knowledge Management Platform** that converts documents into a structured knowledge base similar to Wikipedia, then uses that knowledge to answer questions with better reasoning.

The easiest way to understand it is to compare how humans learn.

- A traditional RAG system is like a student who opens a textbook and searches for paragraphs whenever someone asks a question.
- WikiLLM is like a student who first studies the textbook, organizes the knowledge into notes, connects related concepts, and then answers questions from understanding rather than searching.

That is the fundamental idea behind the project.

1. The Problem This Project Solves

Imagine a company has:

- HR Policies
- Employee Handbook
- Product Manuals
- Technical Documentation
- Meeting Notes
- SOPs
- Research Papers

Thousands of documents exist.

When someone asks:

"How many leave days can interns take?"

Finding the answer manually is slow.

Traditional search requires:

Open document

↓

Search keyword

↓

Read several pages

↓

Find answer

Very inefficient.

Traditional AI (RAG) improves this.

Question

↓

Search vector database

↓

Retrieve paragraphs

↓

LLM answers

Better...

But it still only retrieves text chunks.

It does **not actually understand relationships**.

2. What is Traditional RAG?

RAG means

Retrieval Augmented Generation

Workflow:

Upload PDF

↓

Split PDF into chunks

↓

Create embeddings

↓

Store embeddings

↓

User asks question

↓

Find similar chunks

↓

Send chunks to GPT

↓

GPT answers

Example

PDF

Tesla

Founder:
Elon Musk

CEO:
Elon Musk

Products:
Model S
Model X
Model Y
Cybertruck

Chunks become

Chunk 1

Tesla Founder Elon Musk

Chunk 2

CEO Elon Musk

Chunk 3

Products
Model S
Model X

Each chunk is converted into vectors.

Question

Who founded Tesla?

Vector search retrieves

Founder Elon Musk

GPT answers

Tesla was founded by Elon Musk.

Works well.

But now ask

Which companies founded by Elon Musk manufacture electric vehicles?

Now GPT must connect

Elon Musk

↓

Tesla

↓

Products

↓

Electric Vehicles

Traditional RAG struggles because it only has isolated text chunks.

3. What WikiLLM Does

Instead of storing chunks...

It creates knowledge.

Think about Wikipedia.

Wikipedia page

Tesla

Contains

- Founder
- CEO
- Products
- Industry
- Related Pages

Every page links to other pages.

Tesla

↓

Electric Vehicles

↓

Battery Technology

↓

Lithium-ion Batteries

↓

Renewable Energy

Everything is connected.

WikiLLM automatically creates these pages.

Instead of storing this:

Chunk

Tesla
Founder
Elon Musk

It stores

Entity

Tesla

Founder

Elon Musk

Products

Model S

Industry

Electric Vehicles

Related

Battery Technology

Now the system actually understands relationships.

4. Project Workflow

Complete workflow:

Upload document

↓

Read document

↓

Extract knowledge

↓

Generate wiki pages

↓

Store relationships

↓

Create graph

↓

Create embeddings

↓

User asks question

↓

Retrieve related pages

↓

Reason

↓

Generate answer

Step 1 — Document Upload

Supported files

- PDF
- DOCX
- TXT
- Markdown

Example

Company Handbook.pdf

or

Machine Learning.pdf

Step 2 — Parsing

Parser extracts readable text.

Suppose PDF contains

Introduction

Machine Learning is...

Algorithms

Neural Networks

Decision Trees

Parser outputs

Introduction

Machine Learning is...

Algorithms...

Neural Networks...

Decision Trees...

No AI yet.

Just extracting text.

Step 3 — Knowledge Extraction

Now LLM reads the text.

Example

Tesla was founded by Elon Musk.

Tesla manufactures Model S.

Tesla is headquartered in Texas.

LLM extracts

Entity

Tesla

Entity

Elon Musk

Fact

Founder = Elon Musk

Relationship

Tesla

↓

Founder

↓

Elon Musk

Another relationship

Tesla

↓

Headquarters

↓

Texas

Everything becomes structured.

Example JSON

```
{
  "entity": "Tesla",
  "founder": "Elon Musk",
  "headquarters": "Texas",
  "products": [
    "Model S",
    "Model X"
  ]
}
```

Now GPT has converted text into knowledge.

Step 4 — Wiki Generation

Now create pages automatically.

Tesla.md

Tesla

Overview

Founder

Elon Musk

Products

Model S

Model X

Related

[[Electric Vehicles]]

[[Elon Musk]]

[[Battery Technology]]

Elon Musk page

Elon Musk

Founder of Tesla

CEO

Related Companies

Tesla

SpaceX

Battery page

Battery Technology

Lithium-ion

Electric Vehicles

Tesla

Everything links together.

Exactly like Wikipedia.

Step 5 — Knowledge Graph

Imagine this graph

Tesla

↓

Founder

↓

Elon Musk

↓

Founded

↓

SpaceX

↓

Industry

↓

Aerospace

Instead of paragraphs...

AI has relationships.

Graph databases excel at this.

Step 6 — Storage

Your proposal uses

Markdown

Tesla.md

JSON

Tesla.json

PostgreSQL

Metadata

Document

Pages

Users

Entities

Optional

Vector DB

Embeddings

Step 7 — Embeddings

Even WikiLLM benefits from semantic search.

Every page

Tesla.md

gets embedding.

Question

Tell me about EV manufacturers.

Embedding retrieves

Tesla

BYD

Lucid

Rivian

Step 8 — Question Answering

User asks

Who founded Tesla?

System

Search wiki

↓

Find Tesla page

↓

Retrieve related pages

↓

Find Founder relation

↓

GPT

↓

Answer

Another question

Which companies founded by Elon Musk manufacture electric vehicles?

Now

Question

↓

Find Elon Musk

↓

Find companies

↓

Tesla

↓

Find products

↓

Determine EV company

↓

Answer

This reasoning is far better than isolated chunk retrieval.

Knowledge Graph Example

Imagine

Person

↓

Elon Musk

↓

Founder Of

↓

Tesla

↓

Manufactures

↓

Model S

↓

Category

↓

Electric Vehicle

Every arrow is a relationship.

Real Business Example

Suppose company uploads

HR Manual

Leave Policy

Insurance Policy

Payroll Policy

Employee asks

Can interns take annual leave?

System

Find Leave Policy

↓

Find Employee Type

↓

Intern

↓

Policy

↓

Answer

Another

Does maternity leave affect bonus calculations?

AI follows

Maternity Leave

↓

Payroll Policy

↓

Bonus Rules

↓

Answer

Traditional RAG may retrieve only one document, while WikiLLM can combine information across multiple linked policies.

Technology Stack

Frontend

React

Purpose

- Dashboard
 - Chat UI
 - Upload interface
 - Wiki browser
 - Graph visualization
-

Backend

FastAPI

Purpose

- APIs
 - Authentication
 - File upload
 - Chat
 - Knowledge processing
-

LangGraph

Controls the AI workflow.

Example

Upload

↓

Parser

↓

Knowledge Extraction

↓

Wiki Generation

↓

Store

↓

Embedding

↓

Done

Instead of writing one huge script, each step is a reusable node.

LangChain

Used for:

- Document loading
 - Text splitting (where needed)
 - LLM interaction
 - Embeddings
 - Prompt management
-

PostgreSQL

Stores:

- Users
 - Documents
 - Metadata
 - Entity relationships
 - Chat history
-

ChromaDB

Stores embeddings for semantic similarity search.

Markdown

Stores generated wiki pages in a human-readable, version-controllable format.

PyMuPDF

Reads PDFs efficiently while preserving much of their structure.

Why This Is Better Than Standard RAG

Traditional RAG	WikiLLM
Stores document chunks	Stores structured knowledge
Retrieves by vector similarity	Retrieves by entities and relationships
Weak multi-step reasoning	Strong multi-hop reasoning
Duplicate information across chunks	Reusable knowledge base
Limited understanding of relationships	Explicit knowledge graph
Answers from retrieved text	Answers from organized knowledge plus retrieved context

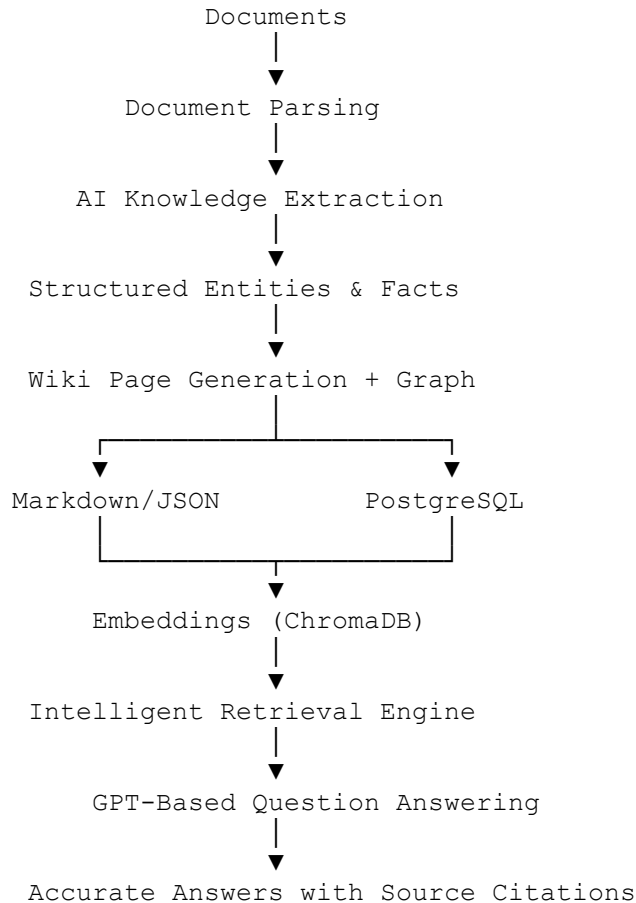
Challenges You'll Need to Solve

Building this system involves several non-trivial engineering problems:

1. **Entity Resolution:** Determine whether "OpenAI", "Open AI", and "OpenAI Inc." refer to the same entity.
 2. **Knowledge Updates:** When a document changes, update existing wiki pages and relationships instead of creating duplicates.
 3. **Conflicting Information:** Handle situations where different documents contain inconsistent facts.
 4. **Citation and Traceability:** Every generated answer should reference the originating wiki pages and source documents.
 5. **Scalability:** Design indexing and retrieval to support thousands or millions of wiki pages.
 6. **LLM Hallucination:** Constrain the model to answer only from extracted knowledge and clearly indicate when information is unavailable.
-

Final Perspective

Conceptually, your system can be summarized as:



In essence, this project is an **AI-powered knowledge management platform**. Rather than treating documents as collections of searchable text, it transforms them into an interconnected knowledge base that supports more accurate retrieval, relationship-aware reasoning, and maintainable organizational knowledge. This aligns with the architecture and objectives outlined in your proposal.